

EPIC — ADO Security Controls Coverage (for Vince)

Quick answers to the ADO Layer 1–5 security questions, based on what EPIC’s pipeline framework and AWS infra actually do today.

Question-by-question

1. Does EPIC enforce OIDC/workload identity federation for AWS access, or stored service connections?

OIDC / workload identity federation — no stored long-lived credentials. EPIC provisions an AWS IAM OIDC provider for Azure DevOps (`vstoken.dev.azure.com/<org-id>`, audience `api://AzureADTokenExchange`). The ADO AWS service connection federates in via `sts:AssumeRoleWithWebIdentity` into `pge-epic-service-role`, scoped by a sub condition (`sc://<org>/<project>/*`). That role then cross-account `sts:AssumeRoles` into each target account’s `pge-epic-deployment-role`. All credentials are temporary session tokens (max 3600s). No access keys in ADO secrets. - EPIC AWS Resources/Terraform State/main.tf (OIDC provider + service role/trust policy) - EPIC AWS Resources/Deploy Role/main.tf (per-account deployment role) - epic-pipeline/infra/aws.yml (assume-role flow)

2. Does EPIC provide environment-level approval gates (separate approvers for prod)?

Partial — gating yes, environment-level approver separation no. Approval is an inline `ManualValidation@0` task (a pool: server stage), conditionally injected when the target environment is listed in `app.approvalEnvironments` in `.pipeline/epic.json`. So you *can* require approval for prod only. But it is **not** backed by ADO **Environments** with per-environment approver/check configuration — there’s a single inline gate and approvers aren’t defined per environment. - epic-engine.yml (Approval stage / `ManualValidation@0`) - epic-orchestrator.yml (`approvalEnvironments` resolution)

3. Are EPIC’s ADO agents ephemeral or persistent?

Mostly ephemeral, with a persistent self-hosted exception. Most stages (download, test, deploy, infra, default build/scan) run on Microsoft-hosted `ubuntu-latest/windows-latest` — fresh VM per job. The exception is the EPIC - Self-hosted pool, used for **.NET + SonarQube builds and Wiz scans** (tools must be pre-installed). That pool is persistent and not refreshed per job. - epic-pipeline/build/main.yml, epic-pipeline/scan/main.yml, epic-pipeline/README.md (Agent Pools)

4. Does EPIC manage its own Marketplace extension allowlist?

No. No extension allowlisting or marketplace governance exists in EPIC. It consumes standard/marketplace tasks (SonarQubeAnalyze@8, JFrogGenericArtifacts@1, AWSShellScript@1, etc.) but does not restrict which extensions are permitted.

5. Does EPIC enforce YAML-only pipelines with branch protection on pipeline definitions?

YAML: yes. Branch protection: not enforced by EPIC. The entire framework is YAML (orchestrator + engine + modular stage templates); there are no classic UI pipelines. However, branch protection on the pipeline definitions themselves is a repo/ADO policy concern and is not something EPIC enforces in-framework.

6. Does EPIC stream ADO audit logs to a SIEM?

No. No SIEM/audit-stream integration. EPIC has rich build **tagging** for traceability (epicRepo, epicAppName, epicEnvironment, epicCloud, epicEngineId), but nothing streams ADO audit logs to Splunk/SIEM.

7. Does EPIC generate SBOMs or sign build artifacts?

No to both. No SBOM generation (no syft/cyclonedx/spdx) and no artifact signing/attestation (no cosign/sigstore/provenance) anywhere in the pipeline. Security scanning *does* exist: SonarQube (SAST + coverage) and Wiz (IaC + secrets + vuln, SARIF output). No container image scanning, dependency-vuln scanning, or DAST.

Summary: provided by EPIC vs. needs the GIS CICD Hub

Control	EPIC native?	Notes
OIDC / workload identity federation to AWS	✅ Provided by EPIC	No stored creds; per-account assume-role chain. The IAM trust fabric itself is EPIC's pge-epic-service-role + deploy-role pattern.
YAML-only pipelines	✅ Provided by EPIC	100% YAML, no classic pipelines.
Vulnerability scanning (SAST/IaC/secrets)	✅ Provided by EPIC	SonarQube + Wiz. Gaps: no container/dependency/DAST scanning.
Approval gates	🟡 Partial	Inline manual gate per approvalEnvironments; not ADO Environment checks with per-env approvers.
Ephemeral agents	🟡 Partial	Hosted stages ephemeral;

Control	EPIC native?	Notes
		EPIC - Self-hosted pool (SonarQube/Wiz) is persistent.
Branch protection on pipeline defs	🟡 Not in-framework	YAML enforced; branch policy is a repo/ADO config, not EPIC.
Marketplace extension allowlist	❌ Not covered	Needs hub-level governance.
ADO audit log → SIEM	❌ Not covered	Tagging only; no SIEM stream.
SBOM generation	❌ Not covered	None.
Artifact signing / provenance	❌ Not covered	None.

Bottom line: Mark **OIDC/identity federation, YAML-only, and SAST/IaC scanning** as “provided by EPIC.” Treat **deployment locks, marketplace allowlist, SIEM audit streaming, SBOM, and artifact signing** as gaps for the GIS CI/CD Hub to own. **Approval gates, ephemeral-agent guarantees, and branch protection** are partial — EPIC has the bones but the hub should layer on ADO Environment checks / scale-set ephemeral agents / branch policy to make them auditable.